

Enrollment Validation Audit

An end-to-end trace of every path that can create an enrollment — what gets checked, what does not, and what would have to move before prerequisite and grade rules can be enforced consistently.

REPOSITORY	BRANCH	REVIEWED	STACK	CODE CHANGED
TMCC-ASRMS	main @ 027b434	2026-08-22	Laravel · PHP 8.2	None

Prerequisite enforcement is not missing — it is unevenly wired

The data model is complete and populated, and a mature rule engine already exists in `AcademicProgressionService`. The real problem is that **three separate write paths create enrollments**, each with a different level of protection, because the rules were re-implemented by hand instead of shared.

That is a materially better starting position than "nothing was built." It changes the scope from *build prerequisite enforcement* to *unify it, wire it in, and fix a crash*.

What actually happens today when a student is enrolled

PATH	ENDPOINT	REACHED FROM	STATE	WHAT IT CHECKS
A	<code>POST /staff/students/{id}/enrollments</code> <code>StudentController.php:468</code>	Staff → Edit Student	CRASHES	Program set, curriculum membership for the chosen year/semester, prerequisites, duplicates — then dies before writing anything.
B	<code>POST /staff/students/{id}/enrollments/add-next-term</code> <code>StudentController.php:1094</code>	Academic Progression (Step 4)	VALIDATED	Term sequencing and residency, curriculum membership, already-passed/credited, active duplicates, prerequisites with AND/OR, unresolved-prerequisite blocking, retake auto-classification, unit-load caps.
C	<code>POST /staff/students</code> <code>StudentController.php:95</code>	Staff → New Student	NO CHECKS	Nothing academic. <code>subject_ids.*</code> is validated only as <code>exists:subjects,id</code> .

Path B is the one that works

It delegates to `validateAddNextTerm()` and writes `Enrollment`, `Grade`, and `EnrollmentAuditLog` together in one transaction. This is the engine worth keeping.

Path C is the open door you described

`StoreStudentRequest` lets any subject in the entire catalogue through — from any program, any year level, ignoring every prerequisite. It also writes no `Grade` rows, no audit log, and hardcodes `year_level => 1`.

Your stated symptom is real and correctly diagnosed — it just lives in student *creation*, not on the enrollment screen. The enrollment screen has the opposite problem: it enrolls nobody at all.

What is missing or wrong

#	SEVERITY	FINDING	LOCATION
a	CRITICAL	Path A is dead code — a fatal <code>TypeError</code> on every single call. Detailed below.	<code>StudentController.php:596–602</code>
b	CRITICAL	Path C performs no academic validation whatsoever. This is the "any subject" hole.	<code>StoreStudentRequest.php:44</code>
c	DIVERGENCE	Path A's prerequisite logic is a third, divergent copy. It ignores <code>prerequisite_logic</code> (treats every OR group as AND) and ignores <code>unresolved_prerequisites</code> entirely — both of which Path B honours.	<code>StudentController.php:537–587</code>
d	DIVERGENCE	Path A's definition of "passed" disagrees with the service. It accepts <code>grade_value</code> 1.00–3.00 or <code>remarks = 'PASSED'</code> (exact case) but never reads the <code>status</code> column, the modern source of truth. A Credited subject with no numeric grade is invisible to it.	<code>StudentController.php:550</code>
e	BYPASS	<code>updateEnrollment</code> re-validates nothing. Staff can change <code>academic_year</code> and <code>semester</code> on an existing row, relocating an enrollment into a term where prerequisites are not satisfied. Narrower than it looks — <code>subject_id</code> is not editable — but it bypasses every check.	<code>StudentController.php:684</code>
f	ABSENT	There is no grade-threshold gate on enrollment anywhere . The only threshold in the codebase is the pass/fail boundary (≤ 3.00) used to satisfy prerequisites. <code>AcademicStandingService</code> computes GWA but is only called on grade entry and for display — never at enrollment time.	<code>AcademicStandingService.php</code>
g	PROCESS	Zero test coverage. <code>tests/</code> contains only the stock Laravel <code>ExampleTest</code> files — which is how a guaranteed-500 endpoint shipped unnoticed.	<code>tests/</code>

Finding (a) in detail — the crash

Line 602 calls `in_array($subjectId, $passedSubjectIds)` inside a closure whose `use` list at lines 596–598 does not import `$passedSubjectIds`. PHP closures do not inherit parent scope, so the variable is always `null`:

```
PHP Warning: Undefined variable $passedSubjectIds
FATAL: TypeError: in_array(): Argument #2 ($haystack)
must be of type array, null given
```

Confirmed empirically on PHP 8.2.12 — matching the `composer.json` `^8.2` constraint — with an isolated reproduction of the same scoping. It is fatal, not a warning. There is no `try/catch` here, unlike `store()`, so it surfaces as an unhandled 500 and the transaction rolls back. `$subjectsToEnroll` is guaranteed non-empty by the check above it, so the loop always runs.

It is actually a two-part bug. `$passedSubjectIds` is only assigned at line 550, *inside* the `if ($prereqEntries->isNotEmpty())` block. So even after fixing the `use` list, it stays undefined whenever no selected subject has a prerequisite. Both halves need addressing.

Where enforcement would hook in

There is **no single choke point today** — that is the core structural issue. `validateAddNextTerm()` is already the real rule engine, but it cannot be reused as-is because it *computes* the term itself via `computeNextAllowedTerm()`, whereas Path A takes an explicit academic year, semester, and year level from a form, and Path C is a brand-new student with no history at all.

The natural hook is to extract the per-subject rule checks into something that accepts `(student, subjectIds, term)` and returns an error list, leaving term *derivation* where it is. That gives three call sites:

PATH	INSERTION POINT	CHANGE
A	StudentController.php:508	Replace the hand-rolled block at lines 537–587 with the shared call.
C	StudentController.php:130	Call inside the <code>store()</code> transaction before the <code>Enrollment::create</code> loop, plus tighten <code>subject_ids</code> in <code>StoreStudentRequest</code> .
B	AcademicProgressionService.php:922	Swap its inline block for the shared call so all three stay in sync.

A standing or GWA gate would be consulted from that same shared validator, so it lands in one place rather than three.

§ 04 · DATA MODEL

Data model gaps that would block implementation

Already in good shape — not blocking

ASSET	STATE	NOTES
<code>curriculum_prerequisites</code>	POPULATED	Many-to-many, properly seeded — roughly 48 prerequisite declarations per program across 3 programs.
<code>prerequisite_logic</code>	PRESENT	AND/OR column, seeded.
<code>unresolved_prerequisites</code>	PRESENT	JSON column for prerequisite codes that could not be mapped; blocks enrollment on Path B.
<code>grades</code> table	SUFFICIENT	Has <code>status</code> , <code>enrollment_id</code> , INC-conversion tracking. <code>getPassedSubjectIds()</code> handles legacy fallbacks cleanly.

The actual gaps

#	SEVERITY	GAP	CONSEQUENCE
1	BLOCKING	Grade rows are only created on Path B. Paths A and C create <code>Enrollment</code> with no matching <code>Grade</code> row.	Every prerequisite check reads the <code>grades</code> table, so subjects enrolled via A or C are invisible to prerequisite logic until someone manually enters a grade. Grade history at enrollment time is path-dependent. This is the one gap that genuinely undermines correctness.
2	INTEGRITY	<code>enrollments.year_level</code> is never set by Paths A or C — nullable, only <code>addNextTerm</code> populates it.	Holes in progression queries that group by year level.

#	SEVERITY	GAP	CONSEQUENCE
3	INTEGRITY	Status vocabulary is not normalised. 'enrolled' vs 'Enrolled', 'PASSED' vs 'Passed'. No shared enum between controller and service.	Lines 615–622 hardcode both casings in a whereNotIn . updateEnrollment restricts status to in:enrolled,completed,dropped , which excludes the capitalised values the service writes.
4	INTEGRITY	grades.enrollment_id has no foreign-key constraint — it is a bare unsignedBigInteger .	Orphaned grade rows are possible.
5	VISIBILITY	Staff have no prerequisite visibility on Path A. programSubjects() eager-loads prerequisite — the deprecated single-column relation the seeder never populates — instead of prerequisites .	The Edit Student curriculum picker shows no prerequisite information at all. Display-only, since the service's legacy fallback is correctly written, but staff get no warning before hitting a rejection.
6	ABSENT	No per-term academic standing record to gate on.	GWA is cached on the student as a single overall figure, so any standing gate has nothing term-scoped to read.

How this was verified, and what it does not cover

- **No running instance.** There is no .env in the repository — it is gitignored, and no SQLite file is present — so the application could not be booted and no real data was queried. Findings come from static tracing plus an isolated PHP reproduction for the crash.
- **The semester type mismatch is a false alarm.** The curriculum schema declares semester as string(20) while the queries compare integers, but the seeders write integers and the UI sends "1st"/"2nd" through a mapping, so it lines up in practice.
- **One residual risk from that mapping.** \$SEMESTER_MAPPING[. . .] ?? 1 at line 505 silently defaults to 1st semester on any unrecognised format. Harmless from the UI; a data-corruption risk for any direct API call.

NOTHING CHANGED ON DISK · REVIEW ONLY

The scoping question to settle first

Is "grade-threshold check" the prerequisite pass/fail rule — already built, just unwired — or a genuine academic-standing gate, which does not exist in any form?

That is the difference between a consolidation task and a new feature, and it is the main thing driving whether this stays At Risk.